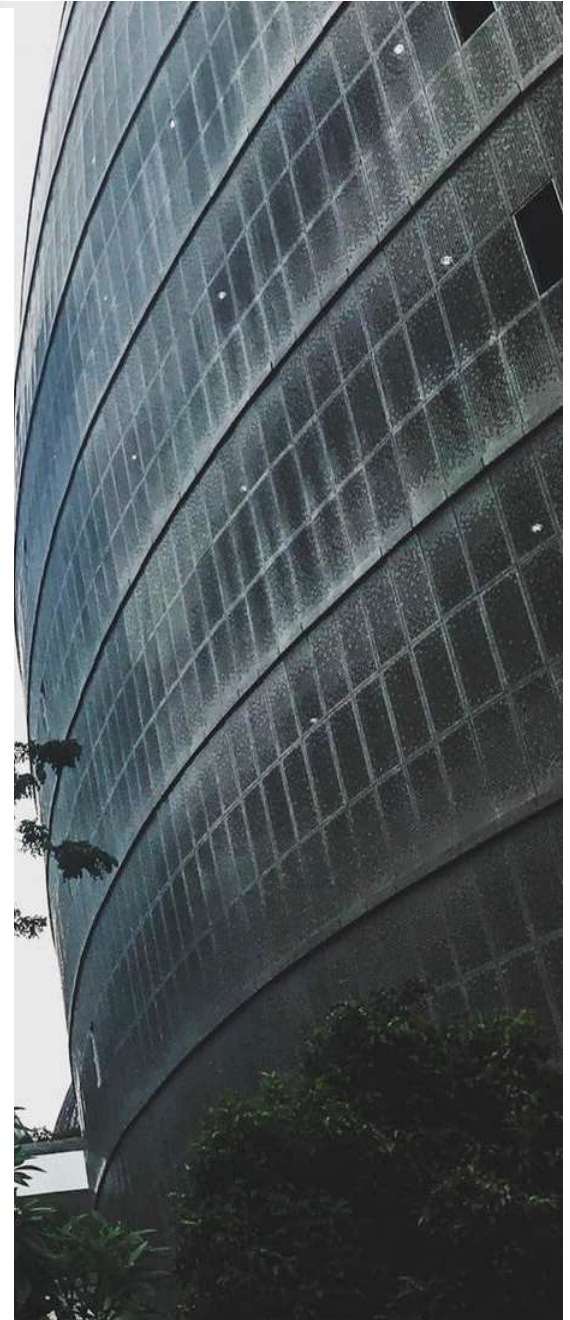


COURSE MATERIAL

**IF670 – Mobile Cross Platform Programming
S1 INFORMATICS GRADUATE PROGRAM
FACULTY OF ENGINEERING AND INFORMATICS**

**INFORMATICS STUDY PROGRAM
FACULTY OF ENGINEERING AND INFORMATION
UNIVERSITAS MULTIMEDIA NUSANTARA**

Gedung B Lantai 5, Kampus UMN
Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia
Telp: +62-21.5422.0808 (ext. 1803), email: lab.fti@umn.ac.id, web: umn.ac.id



MODUL 13

(State Management)

THEME DESCRIPTION

Students will learn how to handle state management in order to avoid props drilling.

WEEKLY LEARNING GOAL (SUB-GOAL LEARNING)

Students will learn how to use Redux for global state management, allowing them to avoid props drilling issues in application development.

SOFTWARE / HARDWARE REQUIREMENTS

1. Laptop / PC
2. IDE (Android Studio/XCode)
3. Code Editor (Visual Studio Code/Sublime/etc)

PRACTICUM STEPS

1. Modify your Week 3 project and install `react-redux`, `@reduxjs/toolkit`, and `redux-thunk`:

`npm install react-redux @reduxjs/toolkit redux-thunk`

2. Create a file named `store.ts`

```
TS store.ts > AppDispatch
1  import { configureStore } from '@reduxjs/toolkit';
2  import rootReducer from './rootReducer';
3
4  // configureStore is a function from Redux Toolkit that simplifies the process of
5  // creating a Redux store.
6  export const store = configureStore({
7    reducer: rootReducer,
8  });
9
10 export type RootState = ReturnType<typeof store.getState>;
11 export type AppDispatch = typeof store.dispatch;
```

3. Create a file named `rootReducer.ts`

```
TS rootReducer.ts > rootReducer > counter
1  import { combineReducers } from '@reduxjs/toolkit';
2  import counterReducer from './counter.slice';
3
4  // combineReducers used to combine multiple reducers into a single root reducer.
5  // In this case, we have only one reducer (counterReducer),
6  // but we can easily add more reducers in the future if needed.
7  const rootReducer = combineReducers({
8    counter: counterReducer,
9  });
10
11 export default rootReducer;
```

4. Create a file named counter.slice.ts



UMN
UNIVERSITAS
MULTIMEDIA
SANTARA

```
TS counter.slice.ts > default
1  import { createAsyncThunk, createSlice, PayloadAction } from '@reduxjs/toolkit';
2
3  // all the state related to the counter feature is defined in this slice,
4  // including the initial state, reducers for handling synchronous actions (increment and decrement),
5  // and extra reducers for handling asynchronous actions (fetchPostsCount).
6
7  export interface CounterState {
8    value: number;
9    status: 'idle' | 'loading' | 'succeeded' | 'failed';
10   error: string | null;
11 }
12
13 const initialState: CounterState = {
14   value: 0,
15   status: 'idle',
16   error: null,
17 };
18
19 // createAsyncThunk is a function from Redux Toolkit that simplifies the process of
20 // creating asynchronous actions.
21 export const fetchPostsCount = createAsyncThunk<number>(<
22   'counter/fetchPostsCount',
23   async () => {
24     const response = await fetch('https://jsonplaceholder.typicode.com/posts');
25     const data = await response.json();
26     return Array.isArray(data) ? data.length : 0;
27   }
28 );
29
30 const counterSlice = createSlice({
31   name: 'counter',
32   initialState,
33   reducers: {
34     increment(state) {
35       state.value += 1;
36     },
37     decrement(state) {
38       state.value -= 1;
39     },
40   },
41   extraReducers: (builder) => {
42     builder
43       .addCase(fetchPostsCount.pending, (state) => {
44         state.status = 'loading';
45         state.error = null;
46       })
47       .addCase(fetchPostsCount.fulfilled, (state, action: PayloadAction<number>) => {
48         state.status = 'succeeded';
49         state.value = action.payload;
50       })
51       .addCase(fetchPostsCount.rejected, (state, action) => {
52         state.status = 'failed';
53         state.error = action.error.message ?? 'Fetch failed';
54       });
55   },
56 });
57
58 export const { increment, decrement } = counterSlice.actions;
59 export default counterSlice.reducer;
```

5. Create a file named hooks.ts

```
TS hooks.ts > ...
1 import { useDispatch } from 'react-redux';
2 import type { AppDispatch } from './store';
3
4 // useAppDispatch is a custom hook that wraps the useDispatch hook from react-redux.
5 // It provides the correct type for the dispatch function, which is AppDispatch.
6 export const useAppDispatch = () => useDispatch<AppDispatch>();
7
```

6. Modify the App.js file

```
JS App.js > App
1
2 import { StyleSheet, View } from 'react-native';
3 import { Provider } from 'react-redux';
4 import { store } from './store';
5 import Counter from './counter';
6
7 export default function App() {
8   return (
9     <Provider store={store}>
10       <View style={styles.container}>
11         <Counter />
12       </View>
13     </Provider>
14   );
15 }
16
17 const styles = StyleSheet.create({
18   container: {
19     flex: 1,
20     backgroundColor: '■ #fff',
21     alignItems: 'center',
22     justifyContent: 'center',
23   },
24 });
25
```




7. Modify the Counter.tsx file

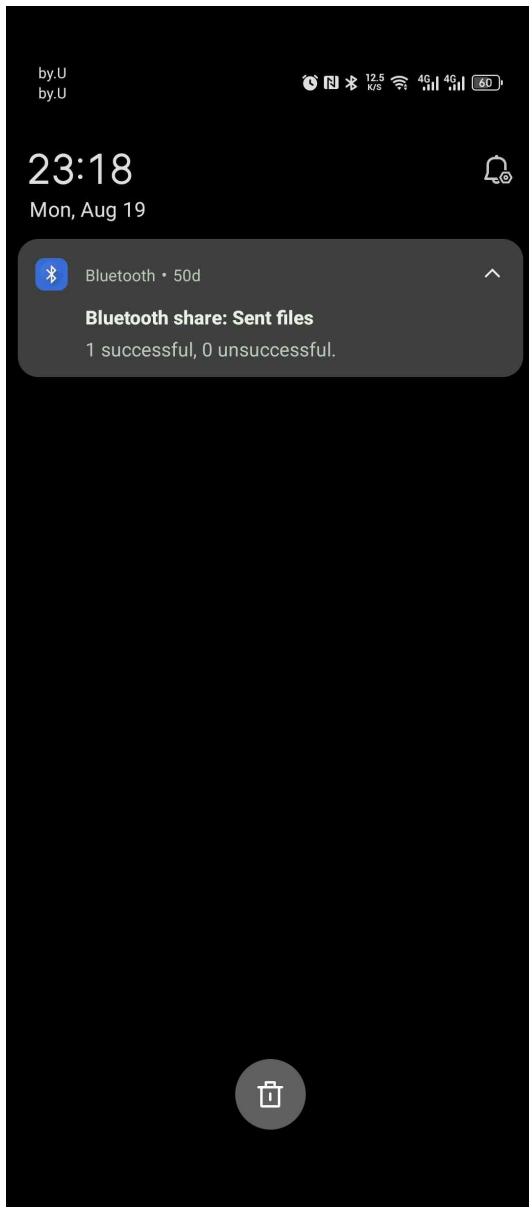
```
counter.tsx > default
1  import { Button, Text, View } from 'react-native';
2  import { useSelector } from 'react-redux';
3  import { useAppDispatch } from './hooks';
4  import { RootState } from './store';
5  import { decrement, fetchPostsCount, increment } from './counter.slice';
6
7  const Counter = () => {
8    const dispatch = useAppDispatch();
9    const count = useSelector((state: RootState) => state.counter.value);
10   const status = useSelector((state: RootState) => state.counter.status);
11   const error = useSelector((state: RootState) => state.counter.error);
12
13   return (
14     <View>
15       <Text>Nama - NIM</Text>
16       {status === 'loading' && <Text>Loading...</Text>}
17       {error && <Text>Error: {error}</Text>}
18       <Text>Count: {count}</Text>
19       <Button title='Increment' onPress={() => dispatch(increment())} />
20       <Button title='Decrement' onPress={() => dispatch(decrement())} />
21       <Button title='Fetch Posts' onPress={() => dispatch(fetchPostsCount())} />
22     </View>
23   );
24 };
25
26 export default Counter;
```

Conclusion:

- store.ts creates the global store from rootReducer (STORE)
- rootReducer.ts combines all slice reducers, including counter (CATALOG)
- counter.slice.ts provides state logic and actions for the counter (SELL/BUY ACTIONS/etc)
- Counter in counter.tsx reads the state and calls actions from the slice
- hooks.ts provides the correct way to use dispatch in TypeScript components

ASSIGNMENT

Modify your Week 12 assignment. Store the total number of successes and failures from every Firebase function that is called (Firestore and FCM). Display these numbers in the notification so that it appears similar to when sending files via Bluetooth, as shown in the image below.



REFERENCES